



Protocol Audit Report

SMART CONTRACT SECURITY ASSESSMENT

AUDITOR **Zakia**
Buras

TARGET CONTRACT
PasswordStore.sol

DATE **August 23,**
2026

CONTENTS

Table of Contents

01	Protocol Summary	3	08	High Severity	5
02	Disclaimer	3	09	H-1	5
03	Risk Classification	3	10	H-2	7
04	Audit Details	4	11	Informational	9
05	Executive Summary	4	12	I-1	9
06	Issues Found	4	13	Conclusion	9
07	Findings	5			

Protocol Overview

Protocol Summary

PasswordStore is a protocol designed to store and retrieve a user's password. The protocol is intended for use by a single user and is not designed to support multiple users.

Only the contract owner should be able to set and retrieve the stored password.

Disclaimer

Every effort has been made to identify as many vulnerabilities as possible within the allocated audit period. However, the auditor assumes no responsibility for vulnerabilities or issues that may remain undiscovered.

This security audit does not constitute an endorsement of the underlying business, product, or protocol. The audit was time-boxed, and the review was focused solely on the security aspects of the Solidity implementation of the contracts within the defined scope.

Risk Classification

Likelihood	High Impact	Medium Impact	Low Impact
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

We use the [CodeHawks severity matrix](#) to determine the severity of findings.

Audit Details

The findings described in this document correspond to the following commit hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/  
└─ PasswordStore.sol
```

Roles

- **Owner:** The only account that should be able to set and retrieve the password.
- **Outsiders:** Any account other than the owner.

Executive Summary

The PasswordStore smart contract was reviewed over a time-boxed period of approximately 1.5 hours. The audit focused on identifying security vulnerabilities in the Solidity implementation, including issues related to access control, data confidentiality, and documentation accuracy.

Issues Found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Gas	0
Total	3

Findings

High

[H-1] Storing a Password On-Chain Makes It Publicly Accessible

Description

All data stored on-chain is publicly accessible and can be read directly from the blockchain.

The `PasswordStore::s_password` variable is declared as `private`, but Solidity's `private` visibility only restricts access from other contracts. It **does not** prevent anyone from reading the variable's value directly from contract storage.

The `PasswordStore::getPassword` function is intended to restrict access to the password to the contract owner. However, this access control does not prevent an attacker from reading the password directly from storage.

One way to read the value stored in `s_password` is demonstrated below.

Impact

Anyone can read the password directly from the blockchain, completely undermining the privacy guarantees of the protocol.

Proof of Concept

The following steps demonstrate how anyone can read the password directly from the blockchain without being the contract owner. We use Foundry's `cast` tool to read the contract's storage directly.

1. Start a Local Blockchain

```
make anvil
```

2. Deploy the Contract

```
make deploy
```

3. Read the Password from Storage

The `s_password` variable is stored in storage slot 1.

```
cast storage <ADDRESS_HERE_OF_SMART_CONTRACT> 1 --rpc-url http://127.0.0.1:8545
```

The command will return output similar to:

```
0x6d7950617373776f7264000000000000000000000000000000000000000000000000000014
```

4. Parse the Stored Value

The hexadecimal value can then be parsed into a string using:

[illegible]

5. Retrieve the Password

The output is:

myPassword

This demonstrates that the password can be retrieved directly from contract storage without calling `PasswordStore::getPassword` or being the contract owner.

Recommended Mitigation

The overall architecture should be reconsidered because storing a plaintext password on-chain cannot provide confidentiality.

One possible approach is to encrypt the password off-chain and store only the encrypted value on-chain. The user would then need to retain an additional secret off-chain to decrypt the password.

Additionally, the getPassword view function should likely be removed or redesigned. Exposing the decrypted password through a public function could result in the password being exposed through transaction calldata or other client-side interactions.

Likelihood & Impact

Metric	Rating
Impact	High
Likelihood	High
Severity	High

[H-2] PasswordStore::setPassword Lacks Access Control, Allowing Anyone to Change the Password

Description

PasswordStore::setPassword is an external function, meaning it can be called by any external account. However, the function does not perform an access-control check to ensure that only the contract owner can call it.

As a result, any user can call setPassword and modify the stored password. This completely undermines the intended functionality of the contract, which states:

“This function allows only the owner to set a new password.”

The vulnerable function is:

```
function setPassword(string memory newPassword) external {  
    // @audit Anyone can call this function because there is no access control.  
    s_password = newPassword;  
    emit SetNetPassword();  
}
```

Impact

Any non-owner user can change the stored password, allowing an attacker to overwrite the owner's password and breaking the intended access-control model of the contract.

Proof of Concept

The following test demonstrates that a non-owner can successfully call setPassword and modify the stored password.

The PasswordStore.t::test_anyone_can_set_the_password test uses Foundry's forge test command to verify that an arbitrary address can call setPassword.

```
function test_anyone_can_set_the_password(address randomAddress) public {
    vm.assume(randomAddress != owner);

    vm.prank(randomAddress);

    string memory expectedPassword = "PasswordAltered";

    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);

    string memory actualPassword = passwordStore.getPassword();

    assertEq(actualPassword, expectedPassword);
}
```

1. Execute the Exploit Test

```
forge test --mt test_anyone_can_set_the_password
```

2. Confirm Successful Exploitation

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 14.39ms (13.24ms CPU time)
```

The test passes, confirming that an arbitrary non-owner address can successfully change the password.

Recommended Mitigation

Restrict PasswordStore::setPassword so that only the contract owner can call it. Implement an access-control check that reverts when msg.sender is not s_owner.

```
if (msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

Likelihood & Impact

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Informational

[I-1] PasswordStore::getPassword Function Signature Does Not Match Its NatSpec Documentation

Description

The NatSpec documentation for PasswordStore::getPassword incorrectly states that the function accepts a newPassword parameter, while the actual function signature is getPassword() and does not accept any parameters.

This discrepancy can cause confusion for developers and users interacting with the contract and makes the documentation inconsistent with the implementation.

Impact

The NatSpec documentation incorrectly indicates that getPassword accepts a parameter, while the actual function does not.

Recommended Mitigation

Remove the incorrect @param newPassword statement and update the NatSpec documentation to accurately reflect the function's behavior.

```
/**
 * @notice This allows only the owner to retrieve the password.
 - * @param newPassword The new password to set.
 */
```

Likelihood & Impact

Metric	Rating
Impact	None
Likelihood	High
Severity	Informational

Conclusion

The audit identified three issues within the reviewed scope:

- 2 High-severity findings

- **1 Informational finding**

The two High-severity findings relate to fundamental security properties of the PasswordStore contract: password confidentiality and access control.

These issues should be addressed before the contract is considered secure for its intended use.

The Informational finding relates to an inconsistency between the NatSpec documentation and the implemented function signature and should be corrected to maintain accurate and reliable documentation.